

RELATÓRIO COMPLETO DE TESTES

Sorriso do Cerrado

Disciplina: Teste de Software

Equipe: Pedro Henrique Cavalcante de Sousa, Samantha Yumi Tanaka, Samuel Batista Rennó, Vinícios Trindade Costa, Wictor Emanuel Pontes Menezes

Professor: Samuel Novais Moura Junior

Data: 19/05/2026

Versão: 1.0

1. Introdução

O presente relatório apresenta os testes realizados no sistema “Sorriso do Cerrado”, desenvolvido como projeto final da disciplina de Teste de Software. O sistema consiste em uma aplicação web de e-commerce voltada para a divulgação e comercialização de produtos artesanais, contendo funcionalidades como autenticação de usuários, gerenciamento de produtos, favoritos, carrinho de compras e painel administrativo.

O objetivo principal deste relatório é documentar os testes executados, os resultados obtidos, os defeitos identificados e as técnicas de teste aplicadas durante o processo de validação do sistema. Os testes foram realizados com foco na verificação de funcionalidades críticas, segurança de acesso, consistência de dados, experiência do usuário e comportamento da aplicação diante de entradas inválidas e cenários inesperados.

Além da execução prática dos testes, foram aplicados conceitos formais estudados em aula, incluindo técnicas de caixa-preta, particionamento de equivalência, análise de valor limite e testes exploratórios, buscando garantir maior cobertura de validação e melhor qualidade do software desenvolvido.

2. Objetivos dos Testes

Os testes executados tiveram como objetivo validar o comportamento funcional e não funcional do sistema “Sorriso do Cerrado”, garantindo que as funcionalidades principais operem corretamente sob diferentes condições de uso.

Entre os principais objetivos dos testes realizados, destacam-se:

CT 001 - Acesso administrativo com usuário cliente

Identificador: CT 001

Tipo: Teste Funcional / Segurança

Descrição: Verificar se um usuário cliente consegue acessar rotas administrativas do sistema.

Passo a passo:

1. Realizar login com usuário cliente
2. Acessar manualmente a rota /admin pela URL
3. Observar o comportamento do sistema

Dados de entrada:

- Usuário cliente autenticado

Resultado esperado:

O sistema deve impedir o acesso à área administrativa.

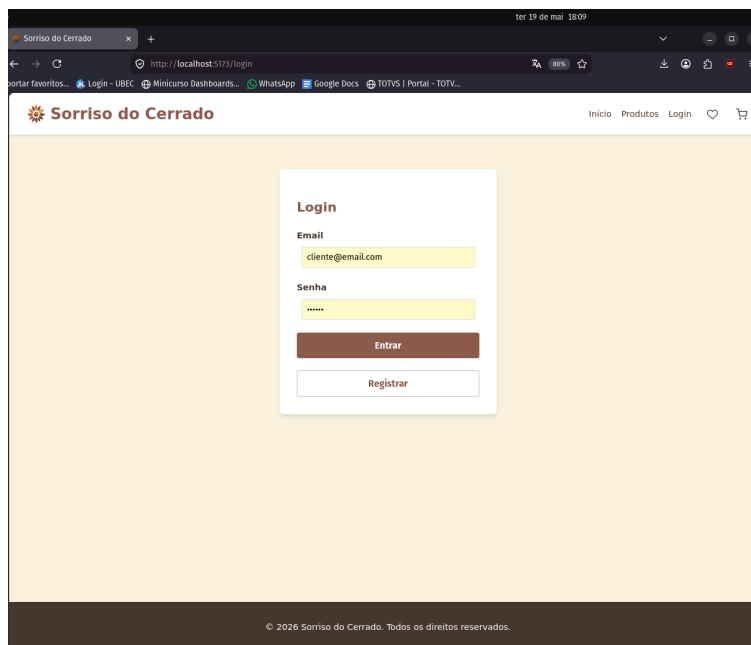
Resultado obtido:

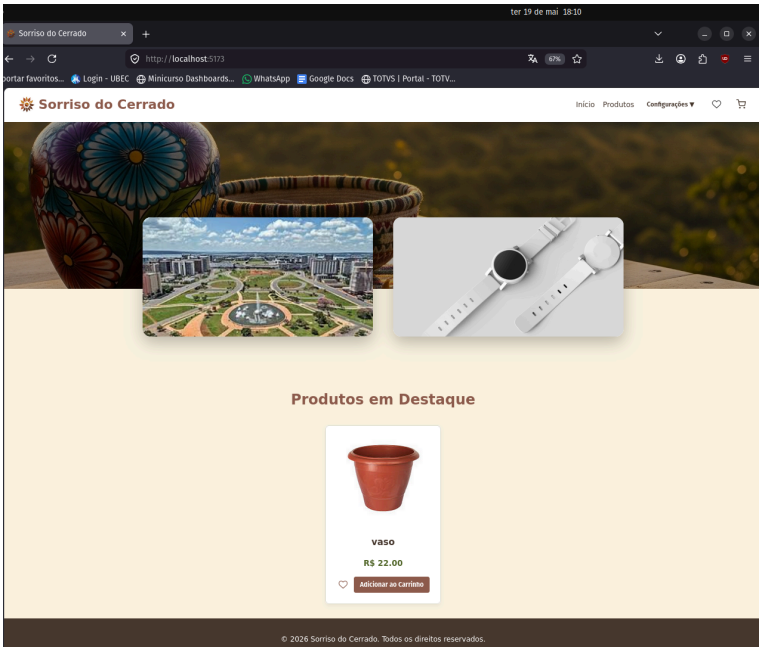
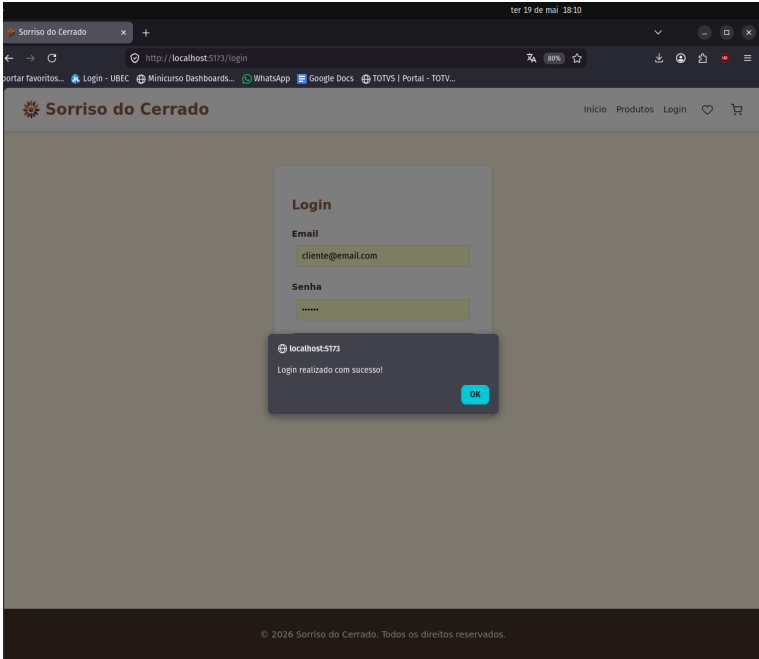
O usuário cliente conseguiu acessar a rota administrativa diretamente.

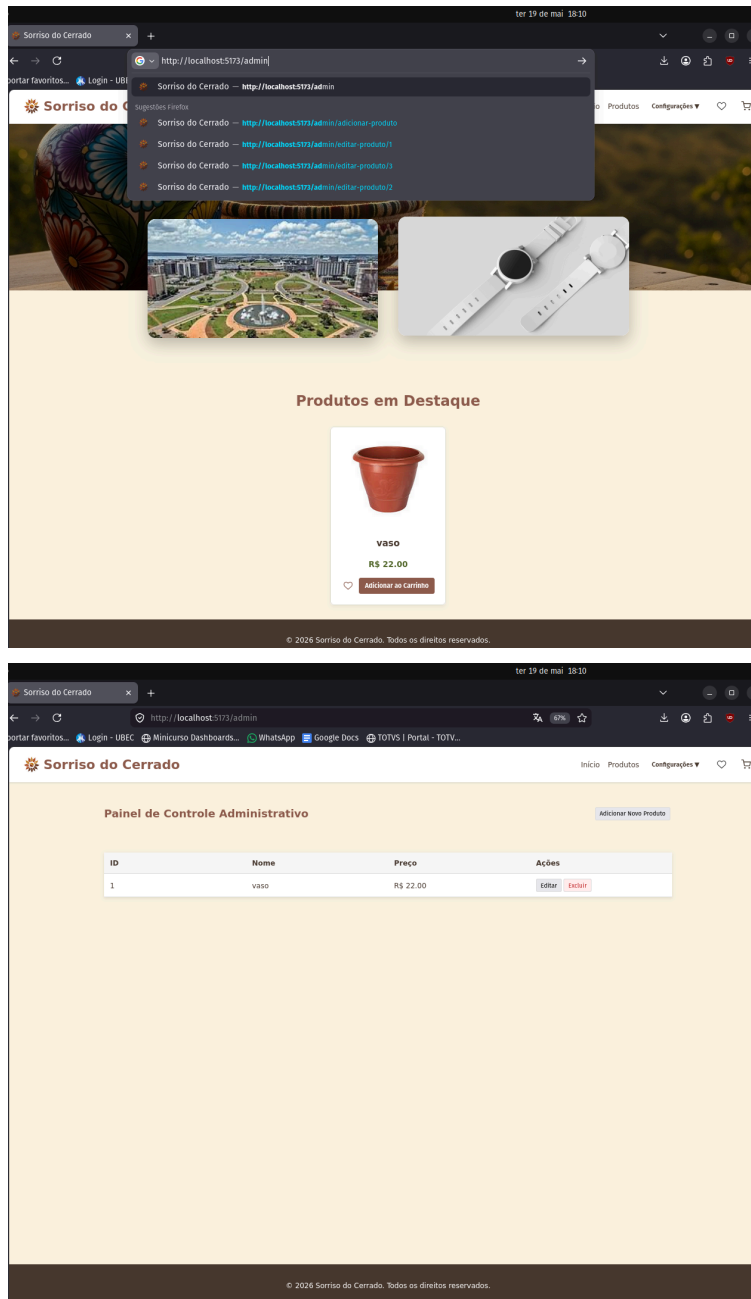
Status:

REPROVADO

Evidência:







CT 002 - Acesso a produto inexistente pela URL

Identificador: CT 002

Tipo: Teste Funcional / Caixa-Preta

Descrição:

Verificar o comportamento da API ao tentar acessar um produto inexistente através de um ID inválido.

Passo a passo:

1. Abrir o Postman

2. Configurar requisição GET para o endpoint de produtos
3. Utilizar um ID inexistente (/produtos/999999)
4. Enviar a requisição
5. Observar a resposta da API

Dados de entrada:

- ID de produto inexistente (ex: 999999)

Resultado esperado:

O sistema deve retornar status HTTP 404 com mensagem indicando que o produto não foi encontrado.

Resultado obtido:

A API retornou status 404 com a mensagem "Produto não encontrado".

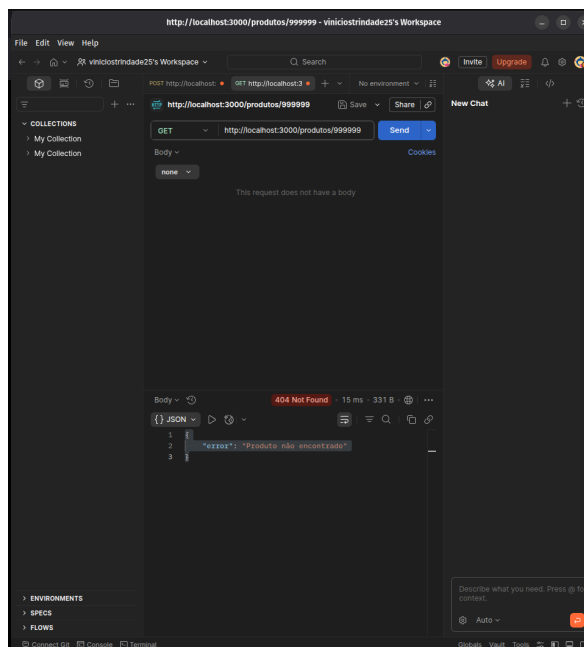
Critério de sucesso:

O sistema deve tratar corretamente requisições para recursos inexistentes, sem falhas ou comportamento inesperado.

Status:

- Aprovado

Evidência:



CT 003 - Atualização da página de favoritos após remoção

Identificador: CT 003

Tipo: Teste Funcional / Exploratório

Descrição:

Verificar se a lista de favoritos é atualizada corretamente após a remoção de um item.

Passo a passo:

1. Realizar login no sistema
2. Acessar a página de favoritos
3. Remover um item da lista

4. Atualizar a página
5. Verificar se o item removido ainda está presente

Dados de entrada:

- Usuário com itens favoritados

Resultado esperado:

O item removido não deve mais aparecer na lista após atualização da página.

Resultado obtido:

O item foi removido com sucesso e não apareceu novamente após atualização da página.

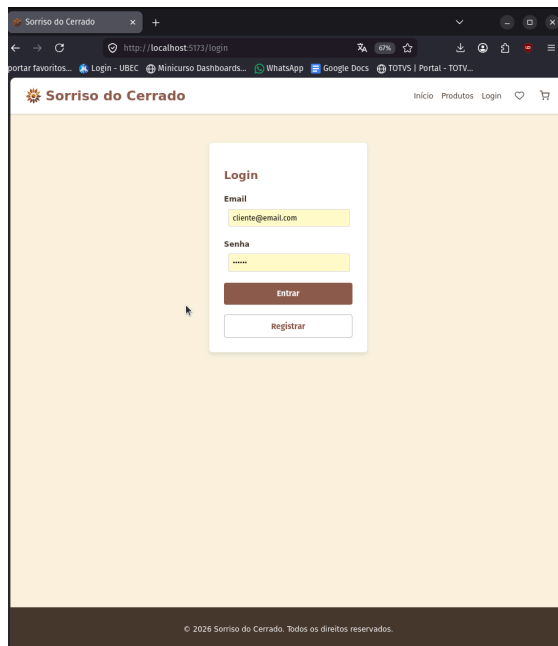
Critério de sucesso:

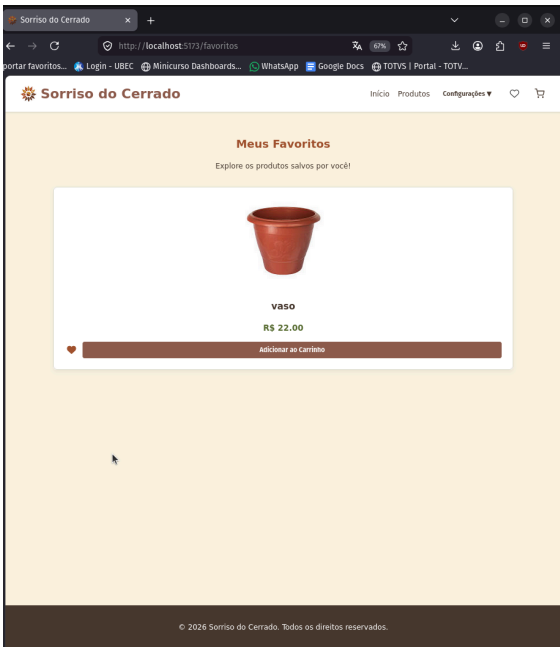
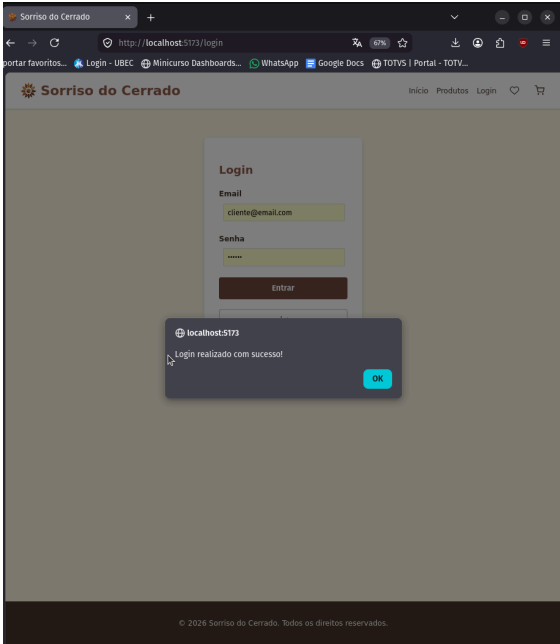
O sistema deve manter consistência entre o estado do backend e a interface do usuário.

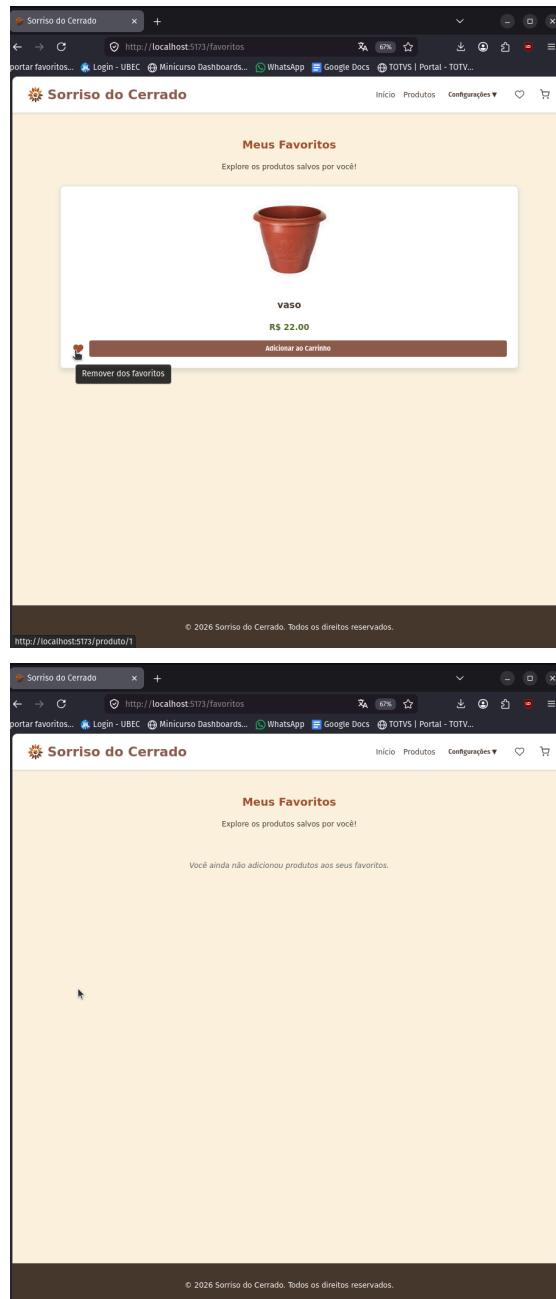
Status:

- Aprovado

Evidência:







CT 004 - Criação de banner via API

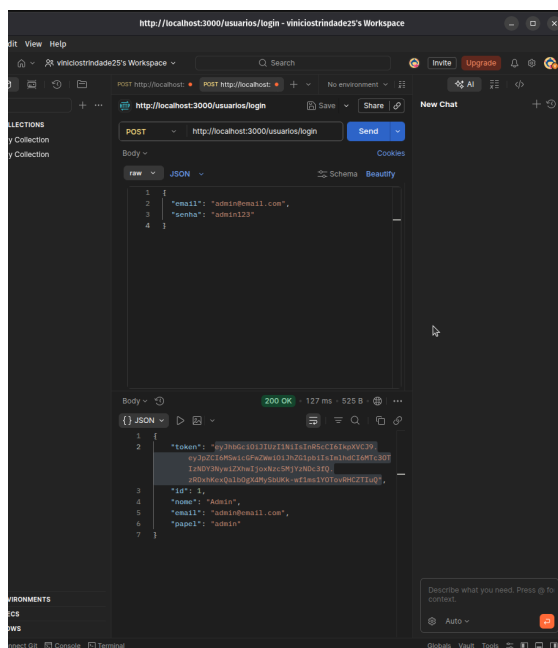
Identificador: CT 004

Tipo: Teste Funcional / Caixa-Preta

Descrição: Verificar o comportamento da API de criação de banners ao receber uma requisição utilizando multipart/form-data, contendo imagem e dados do banner.

Passo a passo:

1. Abrir o Postman
2. Configurar requisição post para o endpoint /banners
3. Adicionar autenticação de usuário administrador (Bearer Token)



4. Cenário 2: enviar requisição com token JWT inválido ou adulterado
5. Observar resposta da API

Dados de entrada:

- Requisição sem header Authorization
- Token JWT inválido ou corrompido

Resultado esperado:

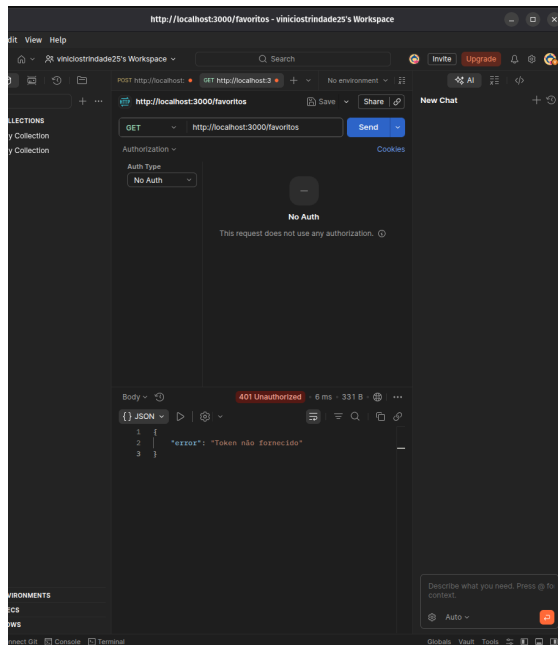
O sistema deve rejeitar ambas as requisições, retornando erro de autenticação (ex: 401 ou mensagem de token inválido/não fornecido), impedindo o acesso ao recurso protegido.

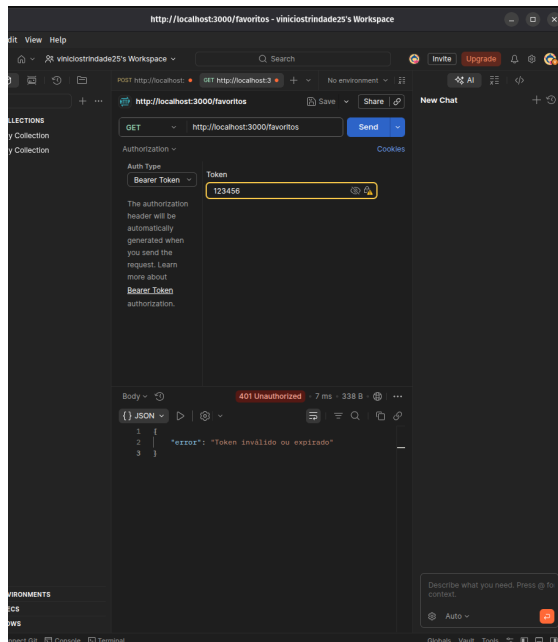
Resultado obtido:

A API bloqueou corretamente o acesso em ambos os cenários. Sem token, retornou erro de ausência de autenticação, e com token inválido retornou mensagem de token inválido ou expirado, impedindo acesso ao recurso.

Status:

- Aprovado

Evidência:



CT 006 - Persistência de autenticação após recarregamento da página

Identificador: CT 006

Tipo: Teste Funcional / Autenticação

Descrição: Verificar se o sistema mantém o estado de autenticação do usuário após recarregar a página, utilizando o token armazenado no localStorage.

Passo a passo:

1. Realizar login com usuário válido (cliente ou administrador)
2. Abrir o sistema normalmente
3. Abrir as ferramentas de desenvolvedor do navegador (DevTools – Firefox)
4. Acessar a aba Storage / Armazenamento → Local Storage
5. Verificar a existência do item token
6. Confirmar que o sistema identifica o usuário como autenticado (menu de usuário ou painel admin no caso de admin)
7. Recarregar a página (F5 ou Ctrl+R)
8. Observar se o estado de autenticação é mantido

Dados de entrada:

Usuário válido com token JWT gerado no login

Resultado esperado:

O sistema deve manter o usuário autenticado após refresh, utilizando o token armazenado no localStorage, sem necessidade de novo login

Resultado obtido:

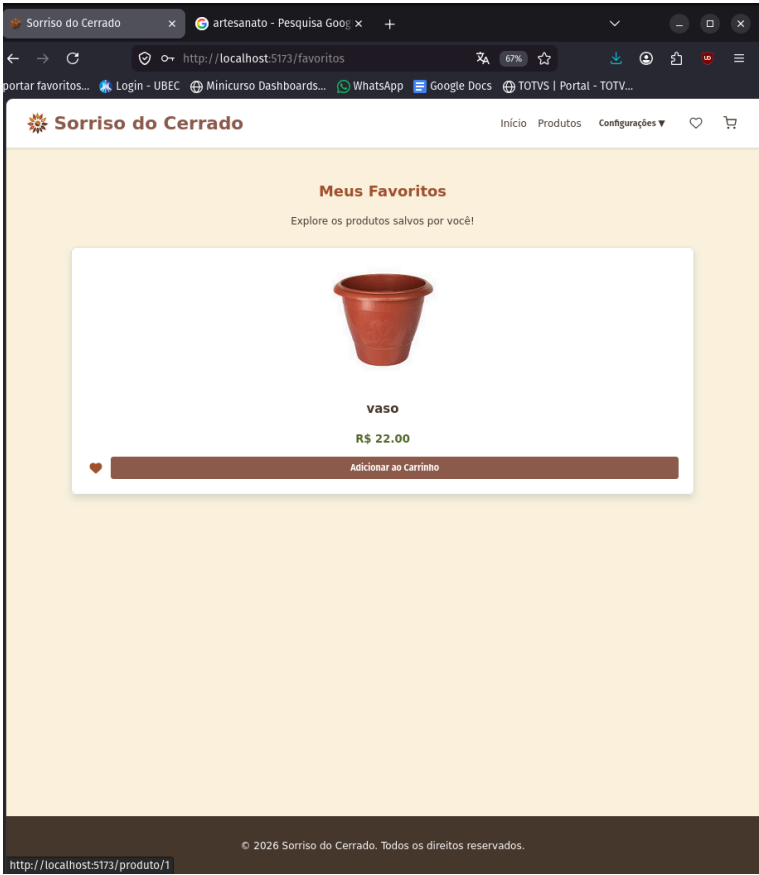
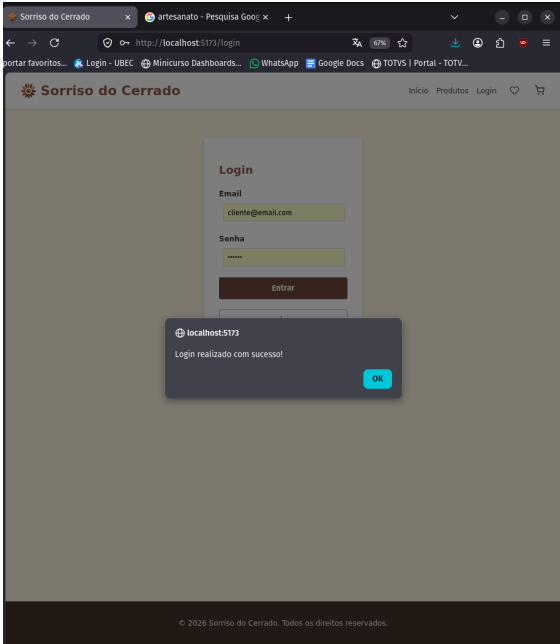
O token JWT foi encontrado no localStorage através do DevTools, e após recarregar a página o sistema manteve o estado de autenticação do usuário conforme o papel definido no token

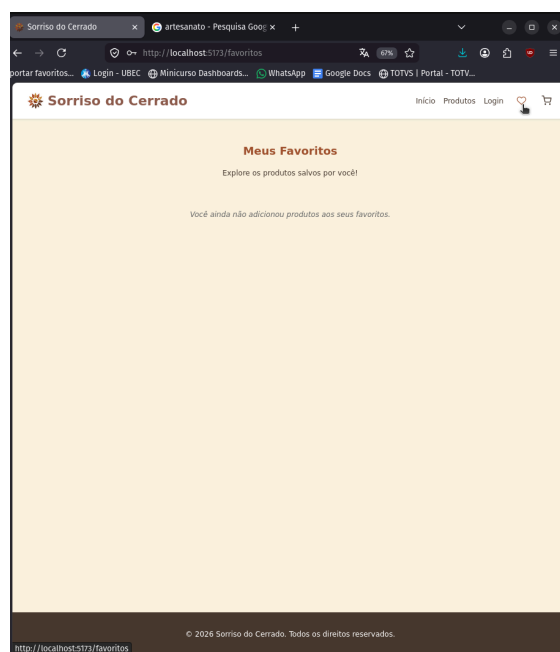
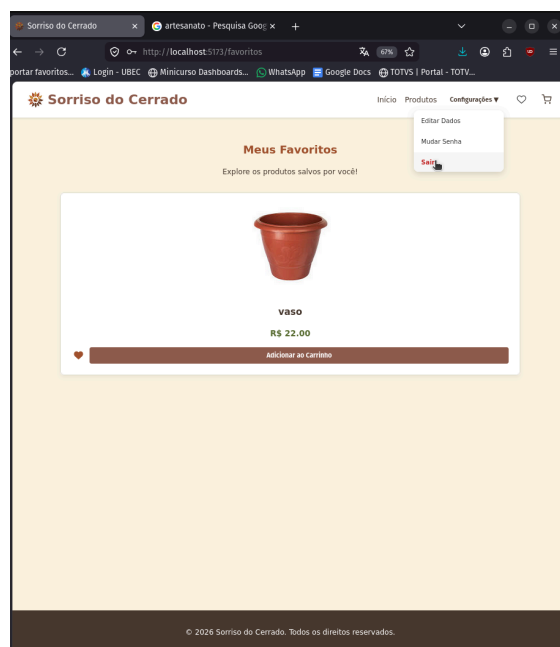
Status:

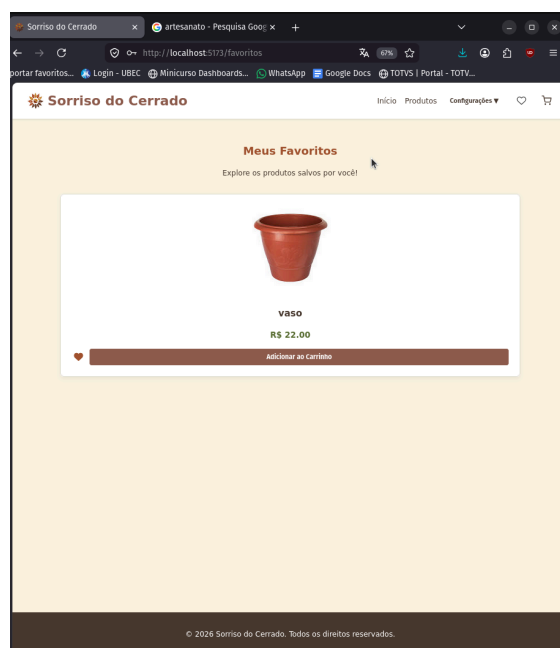
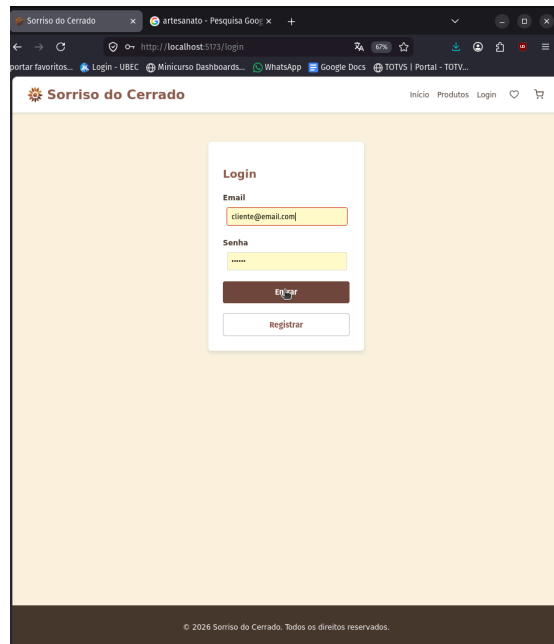
- Aprovado

Evidência:

- Token visível no localStorage via DevTools
- Usuário permaneceu autenticado após atualização da página
- Interface continuou exibindo opções conforme permissão do usuário







CT 007 - Validação de estrutura de e-mail no cadastro de usuário

Identificador: CT 007

Tipo: Teste Unitário / Validação de Dados

Descrição: Verificar se a função interna de validação dos dados do usuário rejeita strings sem os caracteres obrigatórios do email, como o “@”.

Passo a passo:

1. Instanciar uma função para validar e-mail.
2. Tentar realizar o envio de uma string sem o caractere “@” ou sem o domínio final “.com”.

3. Executar o teste.

Dados de Entrada:

- Execução A:
email= "usuario.cerrado.com"
- Execução B:
email= "usuario@cerrado"

Resultado Esperado:

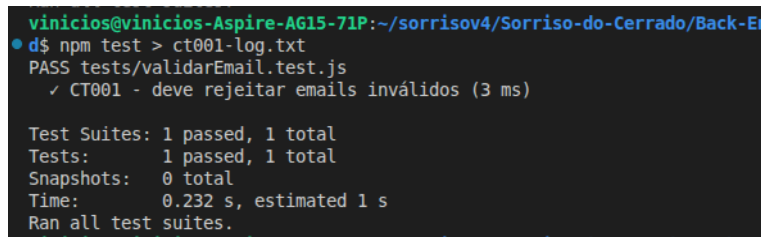
A função deve retornar false para ambos os casos, indicando que o e-mail informado não atende ao formato válido.

Resultado obtido:

A função rejeitou corretamente as entradas inválidas, retornando false para e-mails sem "@" ou sem domínio válido.

Status:

- Aprovado

Evidência:

```
vinicios@vinicios-Aspire-AG15-71P:~/sorrisov4/Sorriso-do-Cerrado/Back-End$ d$ npm test > ct001-log.txt
PASS tests/validarEmail.test.js
  ✓ CT001 - deve rejeitar emails inválidos (3 ms)

Test Suites: 1 passed, 1 total
Tests: 1 passed, 1 total
Snapshots: 0 total
Time: 0.232 s, estimated 1 s
Ran all test suites.
```

CT 008 - Validação de preço positivo no produto

Identificador: CT 008

Tipo: Teste Funcional / Validação de Regras de Negócio

Descrição: Verificar se o sistema impede o cadastro ou atualização de produtos quando o campo de preço recebe valores inválidos, garantindo que apenas valores numéricos maiores que zero sejam aceitos.

Passo a passo:

1. Realizar login com um usuário com perfil artesã ou admin
2. Acessar a funcionalidade de cadastro ou atualização de produto
3. Informar os dados do produto
4. Preencher o campo preço com valor inválido
5. Enviar o formulário
6. Verificar a resposta do sistema

Dados de Entrada:

- Execução A:
produto.nome = "Produto Teste"
produto.preco = 0
- Execução B:
produto.nome = "Produto Teste"
produto.preco = -15

- Execução C:
produto.nome = "Produto Teste"
produto.preco = ""

Resultado Esperado: O Sistema deve rejeitar o cadastros ou atualização do produto em todos os cenários, exibindo mensagem de erro informando que o preço deve ser maior que zero. Nenhum produto com valor inválido deve ser salvo ou atualizado no banco de dados

Resultado obtido:

O sistema bloqueou corretamente valores de preço iguais ou inferiores a zero, impedindo o cadastro ou atualização do produto com dados inválidos.

Status:

- Aprovado

Evidência:

```
PASS src/routes/produtos.test.js
CT-002 - Validação de preço positivo no produto
  ✓ Deve aceitar o produto quando o preço for estritamente positivo (4 ms)
  ✓ Deve recusar o produto quando o preço for igual a zero (1 ms)
  ✓ Deve recusar o produto quando o preço for negativo (7 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        1.05 s, estimated 2 s
Ran all test suites.
PS C:\Users\saman\Desktop\Sorriso-do-Cerrado\Back-End>
```

CT 009 - Autenticação e autorização de acesso às rotas protegidas

Identificador: CT 009

Tipo: Teste de Segurança / Autenticação e Autorização (API)

Descrição: Validar se o sistema controla corretamente o acesso às rotas protegidas, garantindo que apenas usuários autenticados e com papel adequado possam executar operações restritas.

Passo a passo:

1. Realizar login com usuário de papel cliente
2. Copiar o token JWT retornado
3. Fazer requisição POST /banners com o token no header Authorization
4. Verificar resposta do sistema (acesso negado)
5. Realizar login com usuário de papel admin
6. Copiar o token JWT do admin
7. Fazer requisição POST /banners com token do admin
8. Verificar resposta de sucesso da requisição

Dados de Entrada:

- Usuário com papel cliente cadastrado no sistema
- Usuário com papel admin cadastrado no sistema
- Credenciais válidas de login (email e senha)
- Token JWT gerado após autenticação
- Endpoint protegido: POST /banners

Resultado esperado:

O sistema deve retornar erro de acesso negado (403) quando a requisição for feita com o token de um usuário com papel cliente, impedindo a execução da operação. Quando a mesma requisição for realizada com o token de um usuário com papel admin, o sistema deve permitir a execução com sucesso, retornando resposta positiva (200 ou 201) e efetuando a criação do banner.

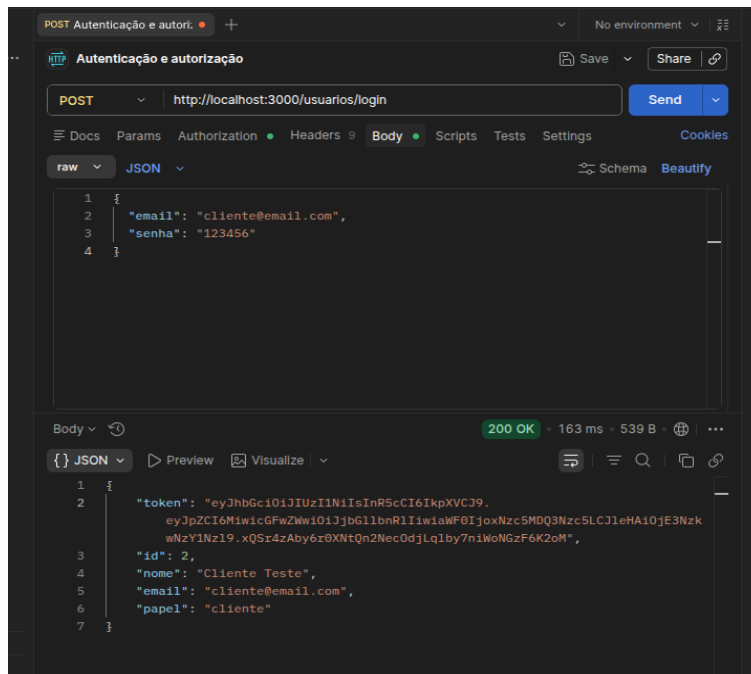
Resultado obtido:

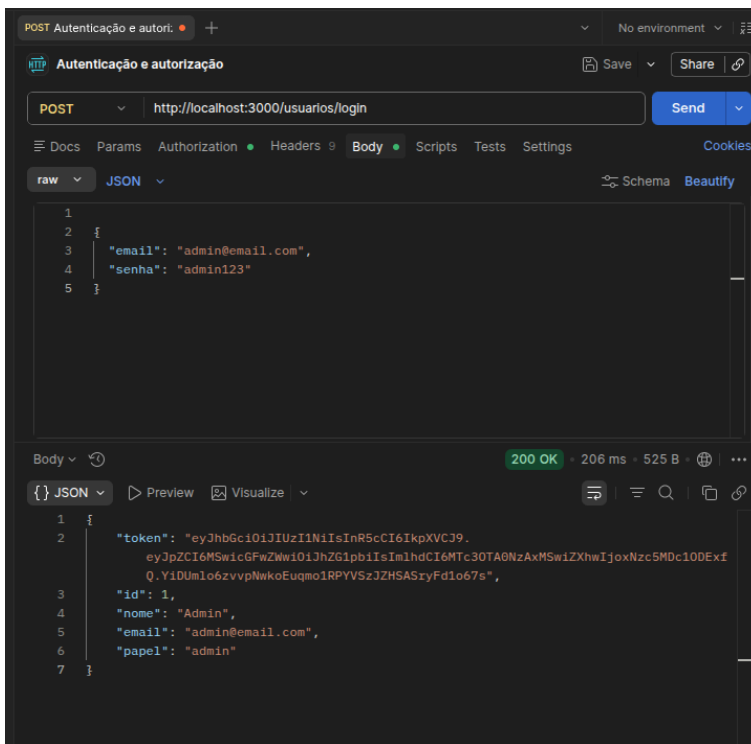
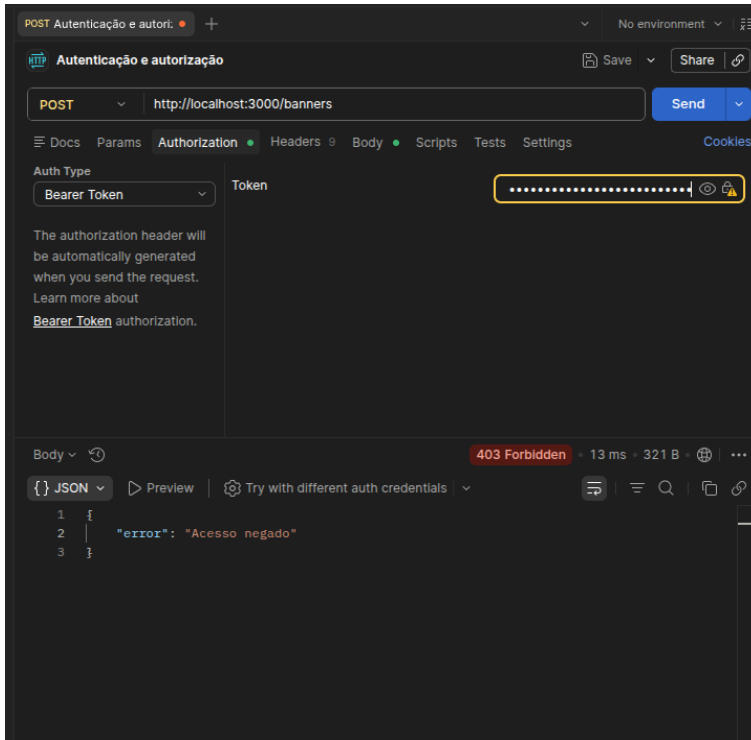
O sistema bloqueou corretamente usuários com perfil cliente ao acessar rota protegida, retornando erro de acesso negado, e permitiu acesso completo para usuário admin.

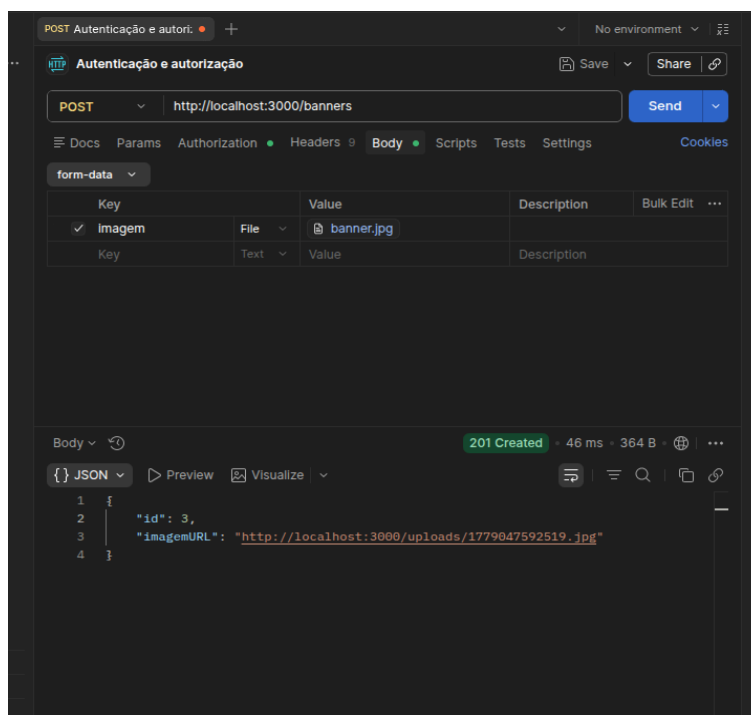
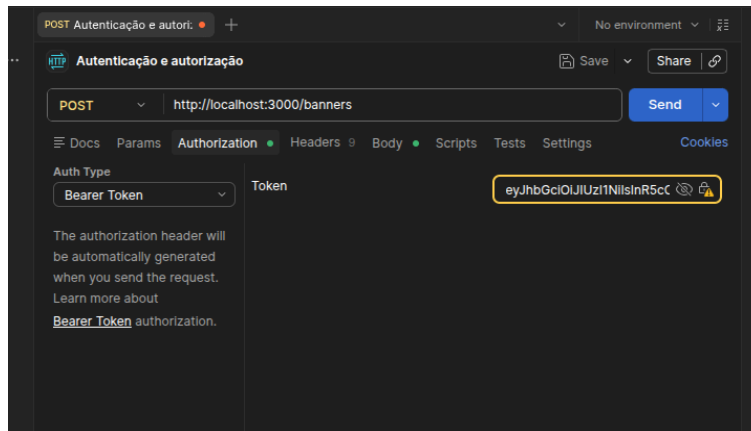
Status:

- Aprovado

Evidência:







CT 010 - Atualização de dados do usuário com validação de unicidade

Identificador: CT 010

Tipo: Teste Funcional / Validação de Dados do Usuário

Descrição: Verificar se o sistema permite que o usuário atualize os seus dados com sucesso. Caso o novo email informado já tenha sido cadastrado, o sistema não deverá concluir a atualização.

Passo a passo:

1. Realizar o login na aplicação com o Usuário A
2. Clicar em configurações e em seguida em “Editar Dados”
3. No campo “Novo email”, digite um e-mail que pertence ao Usuário B, já cadastrado previamente no sistema
4. Clique em “Salvar alterações”

5. Após a mensagem de erro, corrija o campo “Novo email”, por um endereço válido
6. Clique em “Salvar alterações”

Dados de Entrada:

- Execução A (email já existente no banco de dados) :
email= “admin@email.com”
- Execução B (email inédito):
email= “novo@email.com”

Resultado esperado:

Ao tentar salvar as alterações com o email duplicado (Execução A), o sistema deve impedir a atualização. Ao contrário do que deve ocorrer ao salvar as alterações na Execução B, onde o sistema deverá processar a requisição e atualizar o perfil com sucesso.

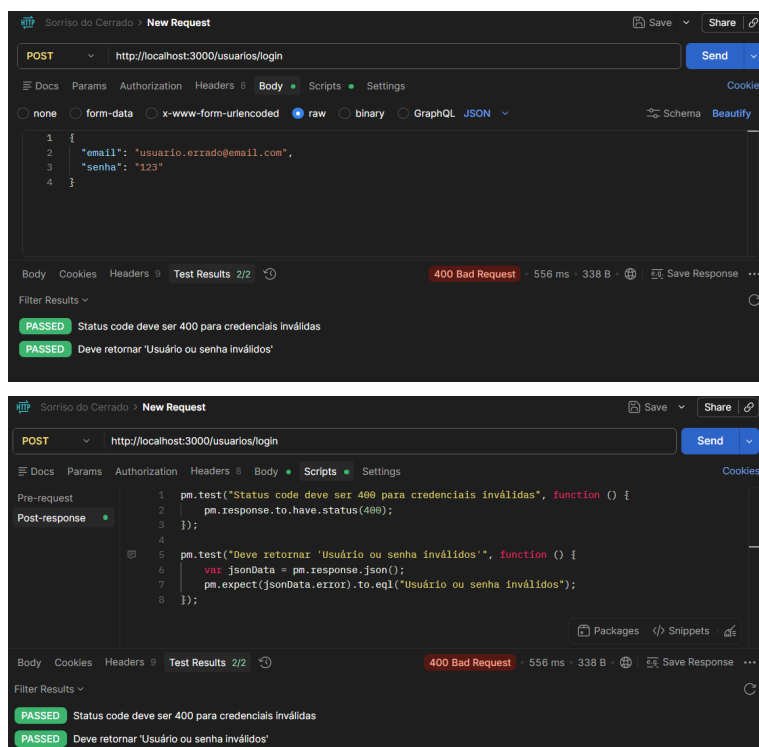
Resultado obtido:

O sistema impediu a atualização do usuário com e-mail já existente e permitiu a atualização após correção para um e-mail válido e único.

Status:

- Aprovado

Evidência:



CT 011 - Produtos favoritos salvos na lista após novo login

Identificador: CT 011

Tipo: Teste Funcional / Persistência de Dados

Descrição: Verificar se os produtos adicionados aos favoritos permanecem disponíveis após o usuário realizar logout e novo login.

Passo a passo:

1. Realizar login no sistema
2. Acessar a lista de produtos
3. Adicionar um produto à lista de favoritos
4. Acessar a página de favoritos e validar presença do item
5. Realizar logout
6. Realizar novo login com a mesma conta
7. Acessar novamente a página de favoritos
8. Validar que o produto anteriormente favoritado ainda está presente

Dados de Entrada:

- Usuário cadastrado com credenciais válidas
- Usuário com pelo menos um produto previamente salvo na tabela de favoritos
- Produto existente no sistema com ID válido e disponível na listagem
- Sistema com banco de dados persistente ativo

Resultado esperado:

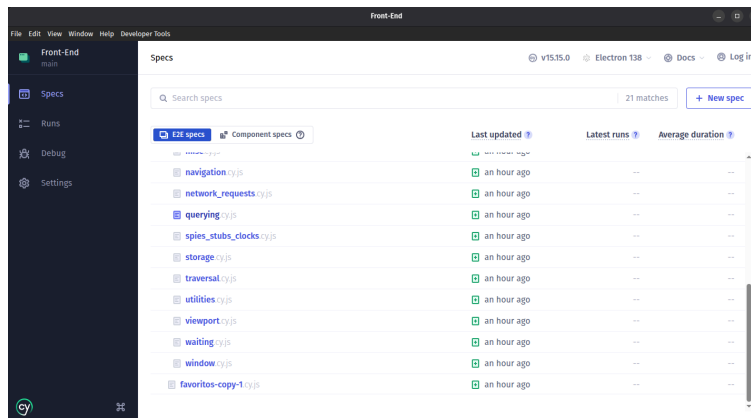
A lista de favoritos carregada corretamente com o produto previamente favoritado ainda listado nos favoritos após o novo login

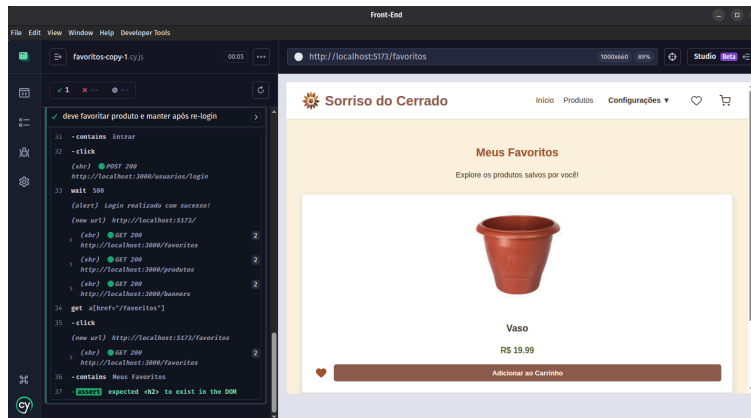
Resultado obtido: Os produtos favoritados permaneceram salvos após logout e novo login, sendo corretamente recuperados do banco de dados e exibidos na interface.

Status:

- Aprovado

Evidência:





CT 012 - Consistência no cálculo e soma do carrinho de compras

Identificador: CT 012

Tipo: Teste Funcional / Cálculo de Carrinho

Descrição: Verificar se o sistema realiza o cálculo aritmético exato sobre o valor total do carrinho de compras, ao adicionar vários itens com preços e quantidades diferentes, assegurando que o valor final seja correto.

Passo a passo:

1. Acessar a vitrine do e-commerce “Sorriso do Cerrado”
2. Adicionar o Produto A no carrinho
3. Adicionar o Produto B no carrinho duas vezes
4. Clicar no carrinho de compras para abrir a tela de pedido
5. Validar se o valor total do carrinho corresponde a soma de todos os produtos

Dados de Entrada:

- Produto A
Quantidade: 1
Preço unitário: R\$45,00
- Produto B
Quantidade: 2
Preço unitário: R\$100,00

Resultado esperado:

O sistema deve exibir no valor total do carrinho “R\$245,00”, e não devem ocorrer arredondamentos ou falhar na soma de valores.

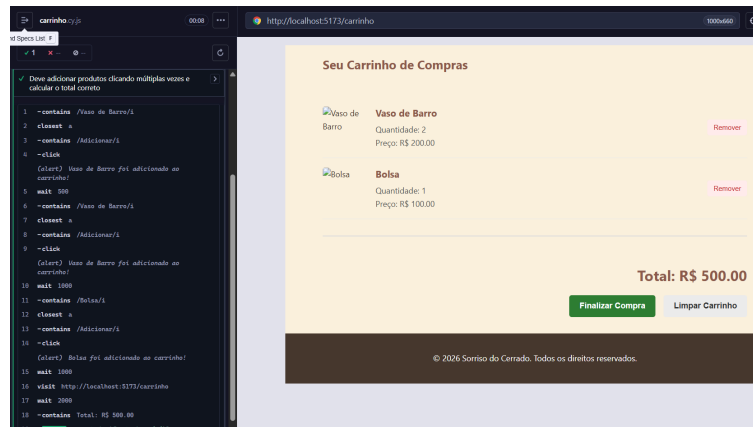
Resultado obtido:

O sistema calculou corretamente o valor total do carrinho, somando os produtos com suas respectivas quantidades sem erros de arredondamento ou inconsistência.

Status:

- Aprovado

Evidência:



CT 013 - Listagem de produtos via Cypress

Identificador: CT 013

Tipo: Teste E2E / Interface

Descrição: Verificar se a página de produtos é carregada corretamente e se os produtos são exibidos na interface através de execução de teste automatizado com Cypress.

Passo a passo:

1. Iniciar o frontend da aplicação
2. Executar o Cypress
3. Rodar o teste automatizado de listagem de produtos
4. O teste acessa a aplicação automaticamente
5. Navega pela interface até a página de produtos
6. Valida se a rota /produtos é carregada corretamente
7. Verifica se os cards de produtos são renderizados na tela
8. Confirma a presença dos elementos visuais dos produtos (imagem, nome, preço e botão de ação)

Dados de entrada:

- Sistema com backend ativo e produtos cadastrados
- Ambiente Cypress configurado e funcional

Resultado esperado:

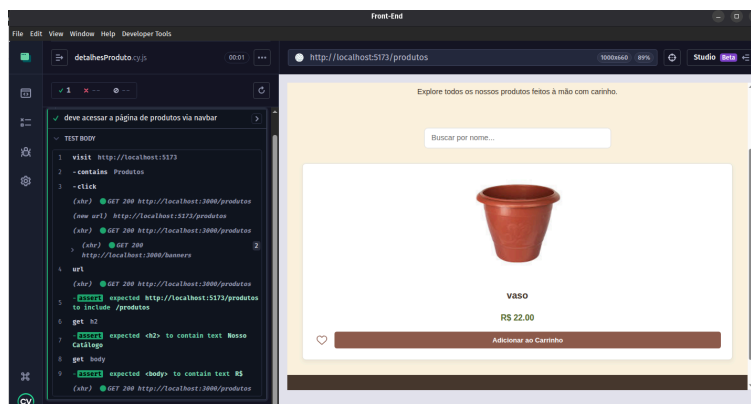
A página de produtos deve ser carregada corretamente, exibindo a lista de produtos em formato de cards, onde cada card contém imagem, nome, preço e botão de ação.

Resultado obtido: O teste automatizado executado via Cypress confirmou que a página foi carregada corretamente, a navegação até a rota de produtos ocorreu com sucesso e os produtos foram exibidos na interface em formato de cards contendo todos os elementos esperados (imagem, nome, preço e botão de ação).

Status:

- Aprovado

Evidência:



CT 014 - Fluxo do carrinho

Identificador: CT 014

Tipo: Teste E2E / Interface

Descrição: Verificar se o sistema permite adicionar produtos ao carrinho e se os itens são corretamente exibidos e mantidos na interface através de execução de teste automatizado com Cypress.

Passo a passo:

1. Iniciar o frontend da aplicação
2. Executar o Cypress
3. Rodar o teste automatizado do fluxo de carrinho
4. O teste acessa a aplicação automaticamente
5. Navega até a página de produtos através da interface
6. Seleciona um produto e aciona a ação de “Adicionar ao Carrinho”
7. Verifica a confirmação da adição do produto ao carrinho
8. Navega até a página do carrinho
9. Valida se o item adicionado está sendo exibido corretamente
10. Confirma a exibição correta das informações do produto (nome, quantidade e preço)

Dados de entrada:

- Sistema com produtos cadastrados no backend
- Usuário acessando o sistema normalmente
- Ambiente Cypress configurado e funcional

Resultado esperado:

Produto deve ser adicionado ao carrinho com sucesso. O sistema deve atualizar o estado do carrinho corretamente. O item deve aparecer na interface do carrinho com dados corretos. A remoção (quando executada) deve atualizar a interface corretamente.

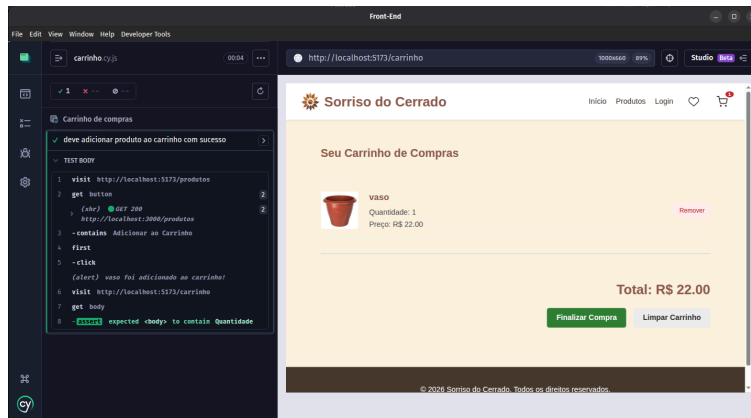
Resultado obtido:

O teste automatizado executado via Cypress confirmou a adição do produto ao carrinho. A navegação pela interface ocorreu corretamente. O item foi exibido corretamente no carrinho com suas informações. O sistema manteve consistência do estado do carrinho durante a execução do teste.

Status:

- **Aprovado**

Evidência:



CT 015 - Validação de adição e incremento de produtos no carrinho

Identificador: CT 015

Tipo: Teste Unitário / Lógica de Carrinho

Descrição: Verificar se o sistema adiciona corretamente produtos ao carrinho quando não existem e, ao adicionar novamente o mesmo produto, incrementa corretamente sua quantidade.

Passo a passo:

1. Executar a suíte de testes unitários com Jest
2. Executar a função responsável pela adição de produtos ao carrinho
3. Observar o resultado da execução no terminal

Dados de entrada:

- Carrinho inicialmente vazio e posteriormente com adição repetida do mesmo produto (id, nome e preço)

Resultado esperado:

O produto deve ser adicionado com quantidade inicial igual a 1 quando novo

Ao ser adicionado novamente, sua quantidade deve ser incrementada corretamente

Resultado obtido:

O sistema adicionou corretamente novos produtos ao carrinho e incrementou a quantidade de itens existentes conforme esperado

Status:

- **Aprovado**

Evidência:

```
vinicios@vinicios-Aspire-A615-71P:~/sorrisov4/Sorriso-do-Cerrado/Back-En
d$ npm test

> backend@1.0.0 test
> jest

PASS tests/carrinho.test.js
  ✓ CT002 - deve adicionar produto novo ao carrinho (5 ms)
  ✓ CT003 - deve aumentar quantidade se produto já existir (1 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 0.261 s, estimated 1 s
Ran all test suites.
vinicios@vinicios-Aspire-A615-71P:~/sorrisov4/Sorriso-do-Cerrado/Back-En
d$
```

CT 016 - Validação de identificação de usuário administrador via token

Identificador: CT 016

Tipo: Teste Unitário / Autenticação

Descrição: Verificar se o sistema identifica corretamente usuários administradores com base no payload do token JWT decodificado.

Passo a passo:

1. Executar testes unitários com Jest
2. Executar função de validação de permissões do token
3. Observar resultado no terminal

Dados de entrada:

- Token JWT contendo payload com papel de usuário (admin ou cliente)

Resultado esperado:

Retornar verdadeiro para usuários admin e falso para usuários não admin

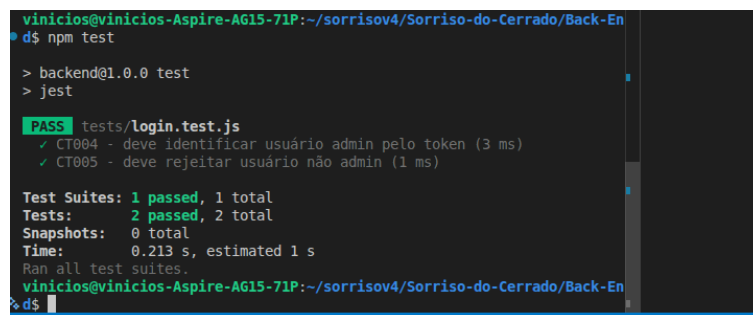
Resultado obtido:

A função identificou corretamente o papel do usuário com base no token

Status:

- Aprovado

Evidência:



```
vinicios@vinicios-Aspire-A615-71P:~/sorrisov4/Sorriso-do-Cerrado/Back-En
d$ npm test

> backend@1.0.0 test
> jest

PASS tests/login.test.js
  ✓ CT004 - deve identificar usuário admin pelo token (3 ms)
  ✓ CT005 - deve rejeitar usuário não admin (1 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 0.213 s, estimated 1 s
Ran all test suites.
vinicios@vinicios-Aspire-A615-71P:~/sorrisov4/Sorriso-do-Cerrado/Back-En
d$
```

3. Ambiente de Teste

Os testes foram executados em ambiente local de desenvolvimento, utilizando ferramentas específicas para testes funcionais, testes de API, testes end-to-end e análise exploratória do sistema.

Tecnologia	Ferramenta
Front-End	React + Vite
Back-End	Node.js + Express
Banco de Dados	Mysql
Teste de API	Postman

Teste E2E	Cypress
Testes Unitários	Jest
Navegador	Chrome/FireFox
Ambiente de Execução	Docker

4. Técnicas de Teste Utilizadas

Durante a execução dos testes foram aplicadas diferentes técnicas de teste de software, visando ampliar a cobertura dos cenários avaliados e aumentar a capacidade de identificação de defeitos no sistema.

Técnica	Aplicação
Caixa-Preta	Validação de entradas e saídas sem acesso à lógica interna do sistema
Particionamento de Equivalência	Agrupamento de entradas válidas e inválidas em classes representativas
Análise de Valor Limite	Testes realizados nos limites mínimo e máximo dos campos de entrada
Testes Exploratórios	Navegação livre no sistema buscando comportamentos inesperados
Suposição de Erros	Execução de testes baseados em falhas comuns em aplicações web
Testes de Segurança	Verificação de autenticação, autorização e controle de acesso
Testes E2E (End-to-End)	Validação completa dos fluxos do usuário desde a interface até o backend

As técnicas utilizadas permitiram avaliar o sistema tanto sob cenários previstos quanto em situações inesperadas, contribuindo para uma análise mais ampla da qualidade da aplicação.

5. Casos de Teste Executados

Os casos de teste foram planejados e executados com foco nas funcionalidades mais importantes do sistema, incluindo autenticação, autorização, persistência de dados,

manipulação de favoritos, validação de entradas e comportamento da aplicação diante de cenários inválidos.

Os testes realizados envolveram diferentes níveis de validação, incluindo testes funcionais, testes de API, testes end-to-end, testes unitários e testes exploratórios.

Entre os principais casos de teste executados, destacam-se:

CT 001 - Acesso administrativo com usuário cliente

Identificador: CT 001

Tipo: Teste Funcional / Segurança

Descrição: Verificar se um usuário cliente consegue acessar rotas administrativas do sistema.

Passo a passo:

1. Realizar login com usuário cliente
2. Acessar manualmente a rota /admin pela URL
3. Observar o comportamento do sistema

Dados de entrada:

- Usuário cliente autenticado

Resultado esperado:

O sistema deve impedir o acesso à área administrativa.

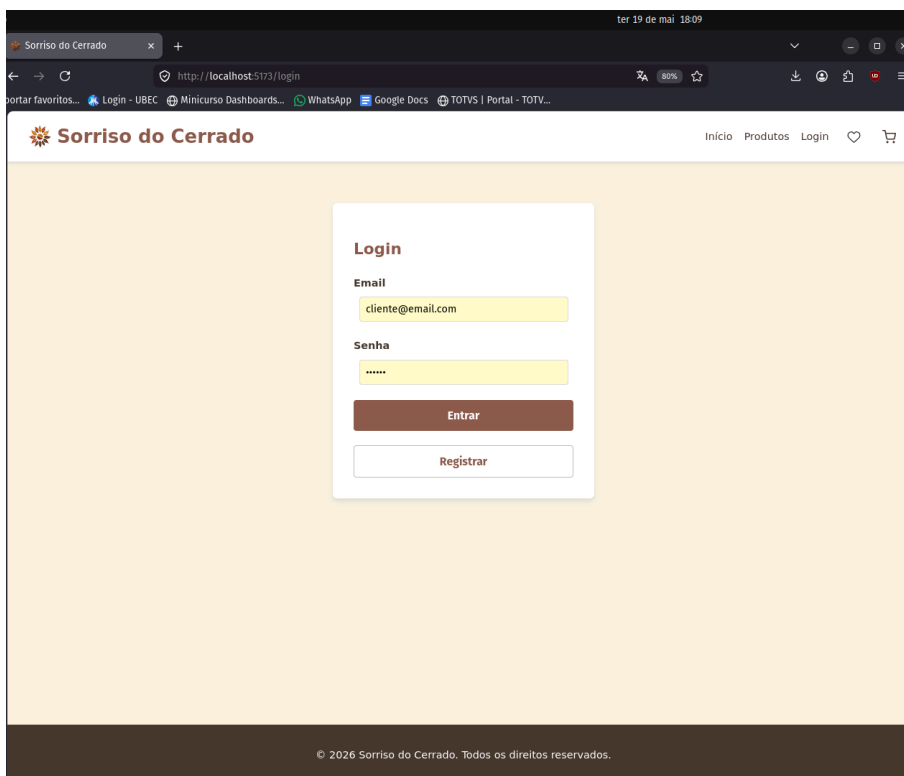
Resultado obtido:

O usuário cliente conseguiu acessar a rota administrativa diretamente.

Status:

REPROVADO ❌

Evidência:



6. Defeitos Identificados

Durante a execução dos testes foram identificados os seguintes defeitos:

- CT 001: Falha na proteção de rota administrativa (usuário cliente acessou área restrita diretamente pela URL)
- Inconsistência inicial em validações de entrada em cenários de dados inválidos (corrigidos durante evolução dos testes)
- Necessidade de reforço no controle de autenticação em rotas protegidas (JWT e autorização por perfil)

Após ajustes e correções no sistema, os demais testes apresentaram comportamento consistente e esperado.

7. Métricas de Teste

- Total de casos de teste executados: 16
- Casos aprovados: 15
- Casos reprovados: 1
- Taxa de aprovação: 93,75%
- Tipos de teste aplicados: Funcional, Segurança, API, Unitário, E2E e Exploratório
- Ferramentas utilizadas: Jest, Cypress, Postman

8. Conclusão

A execução dos testes no sistema “Sorriso do Cerrado” permitiu validar funcionalidades essenciais da aplicação, incluindo autenticação, autorização, manipulação de dados e fluxos de compra.

O sistema apresentou comportamento adequado na maior parte dos cenários, especialmente em testes de interface, persistência de dados e validações de regras de negócio.

Foi identificado um problema de segurança relacionado ao acesso indevido a rotas administrativas, evidenciando a necessidade de reforço no controle de autorização baseado em perfis de usuário.

De forma geral, o sistema demonstra boa estabilidade funcional, necessitando apenas ajustes pontuais em segurança para atingir maior robustez.